

CLAUDE CODE

Proje Yapılandırma Rehberi

CLAUDE.md | Skills | Rules | Agents | Hooks | Commands

Claude Code Yapılandırma Ekosistemi

Claude Code, proje kokundeki yapılandırma dosyalarını otomatik okuyarak her oturumda projeyi anlayan, kurallarına uyan ve araçlarını bilen bir AI asistan haline gelir.

CLAUDE.md

Kalici Hafiza

Proje baglami, mimari,
kodlama standartlari

Skills

Uzmanlik Alanlari

Projeye ozel pattern'ler
ve best practice'ler

Rules

Zorunlu Kurallar

Her zaman uyulmasi
gereken kisitlamalar

Agents

Alt Ajanlar

Uzmanlasmis Claude
ornekleri (review, vb.)

Hooks

Otomasyon

Yasam dongusu olaylarina
bagli shell komutlari

Commands

Slash Komutlari

Ozel /komutlar ile
tekrarlayan gorevler

01 CLAUDE.md — Kalici Hafiza

Her oturum basinda Claude tarafından otomatik okunur. Projenin "beyni" gibi calisir.

Global

~/ .claude/CLAUDE.md

Tum projeler icin gecerli
genel tercihler

Proje

./CLAUDE.md

Proje kokunde, takimla
paylasilir (git)

Dizin

./<alt-dizin>/CLAUDE.md

Alt dizine ozel kurallar
en spesifik = en oncelikli

Ne İçermeli?

Proje aciklamasi ve teknoloji stack'i

Dizin yapisi ve anahtar dosyalar

Sik kullanılan terminal komutlari (build, test, lint)

Kodlama standartlari ve stil kurallari

Mimari kararlar ve design pattern'ler

MCP server yapilandirma notlari

Domain-spesifik terminoloji

Git workflow ve PR surecleri

İPUCU

/init komutu ile Claude otomatik olarak projenizi analiz edip bir CLAUDE.md taslagi olusturur.

02 Skills — Uzmanlık Belgeleri

Claude'a projeye özel pattern'ler ve best practice'ler ogreten Markdown dosyaları.
Claude, description alanına gore ne zaman uygulayacagina semantik olarak karar verir.

Konum ve Yapi

`.claude/skills/<skill-adi>/SKILL.md`

Her skill kendi klasorunde yashar

Klasor adi: kucuk harf, rakam ve tire

500 satirin altinda tutulmalı

Detayli dokuman ayri dosyada referans verilir

YAML Frontmatter

```
---  
name: testing-patterns  
description: Jest test yazarken  
  kullan. TDD workflow icin.  
allowed-tools: Read, Grep  
model: claude-sonnet-4-20250514  
---
```

Nasil Calisir?

1 Tanimla

SKILL.md icinde description alanini zengin anahtar kelimelerle yaz

2 Eslesme

Claude kullanicinin istegini description ile semantik olarak eslestirir

3 Uygula

Eslesen skill otomatik yuklenir, pattern ve kurallar uygulanir

03 Rules — Zorunlu Kurallar

Claude'un her zaman, kosulsuz olarak uymasi gereken kesin kurallar.
Skills'den farki: Rules secimlik degil, her islemde zorunlu olarak uygulanir.

Konum

`.claude/rules/`

Markdown dosyalari olarak tanimlanir

YAML frontmatter ile kapsam belirlenebilir

Her zaman context'e yuklenir (skill gibi secimlik degil)

Yaygin Kural Turleri

Guvenlik

Hassas dosyalara erisim engeli,
secret yonetimi kurallari

Kod Kalitesi

Type hint zorunlulugu,
linter kurallarina uyum

Örnek Rule Dosyası

```
---  
description: Guvenlik kurallari  
globs: "**/*.ts"  
---  
  
# Asla .env dosyalarini commit etme  
# Her API cagrisinda auth kontrolu  
# console.log yerine logger kullan
```

Mimari

Katman bagimlilik kurallari,
import kisitlamalari

Workflow

Branch stratejisi,
test zorunluluklari

Skills vs Rules — Temel Farklar

Ozellik	SKILLS	RULES
Yukleme	Semantik eslesme ile secimlik	Her zaman otomatik yuklenir
Amac	"Nasil yapilir" rehberi	"Ne yapILMAMALI" kisitlama
Kapsam	Belirli gorev turleri icin	Tum islemler icin gecerli
Konum	.claude/skills/<adi>/SKILL.md	.claude/rules/<adi>.md
Model Atama	Farkli model atanabilir	Model atama yok
Davranis	Onerici (best practice)	Zorlayici (must / must not)

Ornek ile Anlayalim

SKILL ornegi:

"Test yazarken AAA pattern kullan, mock'lar icin factory function olustur, coverage %80'in altina dusmesin" -> Sadece test yazarken aktif olur

RULE ornegi:

"Asla .env dosyasini okuma/duzenleme, console.log kullanma, any type kullanma" -> Her islemde, her zaman kontrol edilir

04 Agents — Uzmanlaşmış Sub Agentlar

Kendi context window'u ve rolü olan özel Claude örnekleri. Belirli görev türleri için daha iyi sonuç ve daha az token tüketimi sağlar.

Konum ve Format

`.claude/agents/<agent-adi>.md`

YAML frontmatter + Markdown içerik

Görev esleştiginde Claude otomatik çağırır

Örnek Agent

```
---
name: reviewer
description: Kod review için
model: sonnet
color: orange
---
```

Sen uzman bir code reviewer'sin.

Örnek Agent Roller

Code Reviewer

Güvenlik, performans ve sürdürülebilirlik odaklı kod incelemesi yapar

sonnet

Debugger

Hata analizi ve root cause tespiti için uzmanlaşmış ajan

sonnet

Architect

Mimari kararlar ve sistem tasarımı için derinlemesine analiz

opus

Tester

Unit test ve integration test yazımı için uzmanlaşmış ajan

haiku

NOT

Agent Teams (deneysel) ile birden fazla ajan paralel çalışabilir. CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS ile aktif edilir.

05 Hooks — Yaşam Döngüsü Otomasyonu

Belirli olaylarda otomatik çalışan shell komutları. Prompt'tan bağımsız, deterministik çalışır. Kod formatlama, güvenlik kontrolü, bildirim gibi işlemleri garanti altına alır.

Hook Olayları (Lifecycle Events)

Oturum Yaşam Döngüsü

SessionStart	Oturum başlarken
UserPromptSubmit	Prompt gönderilmeden
PreCompact	Compaction öncesi
SessionEnd	Oturum kapanırken

Arac Yaşam Döngüsü

PreToolUse	Arac öncesi
PermissionRequest	İzin istegi
PostToolUse	Arac sonrası
PostToolUseFailure	Başarısızlık

Ajan ve Diğer

Stop	Yanıt bitince
SubagentStart	Alt ajan başlangıcı
SubagentStop	Alt ajan bitisi
Notification	Bildirim

Hook Tipleri

command

Shell komutu çalıştırır
Exit code: 0=OK, 2=engelle

prompt

LLM (Haiku) ile değerlendirme
JSON decision dondurulabilir

agent

Alt ajan tabanlı hook
Karmaşık değerlendirmeler için

05b Hooks — Pratik Ornekler

Otomatik Formatlama

PostToolUse

```
matcher: "Edit|Write"  
command: "prettier  
  --write $CLAUDE_FILE_PATHS"
```

Dosya her duzenlediginde
otomatik prettier calisir

Guvenlik Engeli

PreToolUse

```
matcher: "Edit|Write"  
command: python3 -c  
  # .env icin exit(2)
```

.env ve .git/ dosyalarina
yazmayi engeller

Branch Korumasi

PreToolUse

```
matcher: "Edit|Write"  
command: "[ $(git branch  
  --show-current) != main ]"
```

main branch uzerinde
duzenlenmeyi engeller

Bildirim

Notification

```
command: "notify-send  
  'Claude'  
  'Girdiniz bekleniyor'"
```

Claude input beklerken
masaustu bildirimi
gonderir

06 Commands — Özel Slash Komutları

Tekrarlayan görevleri tek bir /komut ile çalıştırmak için kullanılır.
Doğal dil ile yazılır, \$ARGUMENTS parametresi desteklenir.

Proje Komutları

`.claude/commands/<ad>.md`

Takımla paylaşılan projeye özel komutlar

Kişisel Komutlar

`~/.claude/commands/<ad>.md`

Tüm projelerde geçerli kişisel komutlar

Örnek Komutlar

/fix-issue

\$ARGUMENTS ile çalışır:
/fix-issue 123
Kodlama standartlarına
göre issue düzelten komut

```
Fix issue #$ARGUMENTS  
following our coding  
standards
```

/commit

Git context ile zengin
otomatik commit mesajı
oluşturur

```
## Context  
- !`git status`  
- !`git diff HEAD`  
Create meaningful commit
```

/review

Mevcut dosya veya PR
üzerinde kapsamlı kod
incelemesi başlatır

```
Review this code for  
security, performance  
and maintainability
```

Tam Proje Yapılandırma Ağacı

```
proje-koku/  
  CLAUDE.md          <- Proje hafizasi  
  .claude/  
    settings.json     <- Hook + izin  
    settings.local.json <- Kisisel  
    skills/  
      testing-patterns/  
        SKILL.md  
      api-conventions/  
        SKILL.md  
    rules/  
      security.md  
      code-quality.md  
    agents/  
      reviewer.md  
      debugger.md  
    commands/  
      fix-issue.md  
      commit.md
```

Renk Kodlari



CLAUDE.md

Kalici hafiza, her oturumda okunur



Skills

Semantik eslesen uzmanlik belgeleri



Rules

Her zaman zorunlu kurallar



Agents

Uzmanlasmis alt ajanlar



Hooks

settings.json icinde tanimlanir



Commands

Slash komutlari (/komut)

GLOBAL YAPILANDIRMA

```
~/.claude/CLAUDE.md    Tum projeler  
~/.claude/settings.json Tum ayarlar  
~/.claude/commands/    Kisisel komutlar
```

Sonuç

Başlarken Önerilen Adımlar

1

CLAUDE.md Oluştur

/init ile başlayıp, proje stack'i, izin yapısı ve komutları ekleyin

2

Rules Tanımla

Güvenlik ve kod kalitesi için zorunlu kuralları ekleyin

3

Skills Ekle

Projeye özel pattern'ler için SKILL.md dosyaları yazın

4

Hooks Kur

Formatter, linter ve güvenlik kontrollerini settings.json'a ekleyin

5

Commands Yaz

Tekrarlayan görevler için özel slash komutları oluşturun

6

Agents Tanımla

Özel roller için uzman ajanları ekleyin